

Assignment 1

Packet Sniffing & Spoofing

Network Security (50.020)

0. Objectives

In this lab, you will learn:

- Fundamental techniques for network packet sniffing and traffic inspection.
- Identification and interpretation of key IP packet fields.
- Controlled network traffic generation using Python and Scapy.
- Principles and mechanisms of source-IP spoofing.
- Analysis and comparison of normal and spoofed network traffic.
- Extraction of packet-level features from captured traffic.
- Application of AI techniques for distinguishing normal and spoofed traffic.
- Evaluation of AI-based detection results and their limitations.

Lab Setup:

- VirtualBox
- 1 Kali VM (root:toor) + NAT Network
- 3 Docker containers + Internal Network

1. Generate Network Traffic

Network communication relies on protocols that define how data is exchanged between devices. In this task, you will generate several common types of network traffic between two Kali Linux virtual machines.

- **Ping (ICMP):** Used to test basic network connectivity. It sends ICMP Echo Request messages and expects Echo Reply messages from the destination.
- **Telnet (TCP):** Provides remote terminal access, typically over TCP port 23. Telnet does not encrypt transmitted data, making it unsuitable for secure remote administration.
- **SSH (TCP):** Provides secure remote terminal access, typically over TCP port 22. SSH encrypts the communication and is widely used for remote system administration.

This lab uses one Kali Linux VM as the host system. To reduce resource usage while still providing multiple independent hosts, Docker containers are used. Docker provides lightweight isolated Linux environments, each with its own IP address, processes, interfaces, and services.

On the Kali, install the required packages:

```
➤ sudo apt update && sudo apt install -y docker.io docker-compose xauth
```

Copy **lab_setup.zip** to the Kali Desktop, extract it, enter the extracted folder, and start the environment:

- unzip lab_setup.zip
- cd lab_setup
- chmod +x *
- ./start.sh

The script creates the network, starts the containers, configures the services, and performs basic checks. You can also run `./stop.sh` to delete containers and clear the environment after this lab. Use the following command to view the running containers:

- `docker compose ps`

To enter a container directly as root, press `Ctrl+Alt+T` to start a new terminal, then:

- `docksh node1`
- `docksh node2`
- `docksh node3`

First, identify the IP addresses of each containers/nodes and confirm they are connected.

- `ip addr`
- `ifconfig`

From node1, send 5 ICMP Echo Request messages to node2 and observe the replies. Take a screenshot showing the successful Ping result, including the destination IP address and packet statistics.

- `ping -c 5 10.x.x.x`

Question: If the Ping command succeeds, what does this confirm about communication between node1 and node2? Does it necessarily mean that all network services on node2 are accessible?

Now, establish a Telnet session from node1 to node2 and execute several basic commands after logging in. Take a screenshot showing the successful Telnet connection and the command output. Run telnet on node1: (Credentials: root:toor)

- `telnet 10.x.x.x`

After logging in, you can type any commands you want. For example, create an empty folder. Type `"exit"` in the same terminal to terminate the session after completing your exploration.

Establish an SSH connection from node1 to node2 and execute some basic commands. Take a screenshot showing the successful SSH connection and command output. SSH service has been configured. Run SSH on node1:

- `ssh kali@10.x.x.x`

Type `"exit"` in the same terminal to terminate the session.

Questions: Telnet and SSH both support remote terminal access. What is the main security weakness of Telnet, and why is it generally unsuitable for modern remote administration?

2. Packet Sniffing and Traffic Analysis

Packet sniffing allows network traffic to be captured and inspected for troubleshooting and security analysis. In this task, you will first use **tcpdump**, a command-line packet capture tool that is lightweight and suitable for systems without a graphical interface. You will then use **Wireshark** to examine the same traffic in greater detail through a graphical interface.

On node2, identify the network interface connected to the NAT Network, then start tcpdump on that interface. While tcpdump is running, generate Ping, Telnet, and SSH traffic again from node1. Observe how the captured packets differ for each protocol. Take a screenshot showing representative packets captured by tcpdump.

```
➤ tcpdump -i <interface> -nn -v # Stop the capture with Ctrl+C
```

Questions: What information can you identify directly from the tcpdump output? Write and run the appropriate tcpdump filter commands to capture only Ping, Telnet, and SSH traffic, and attach screenshots showing the filtered packets for each command. Can you see the password you entered using Telnet and SSH from network traffic? If yes, attach screenshots; if no, why?

Wireshark provides a graphical interface for inspecting captured packets and analysing individual protocol fields. You can use tcpdump with the -w option to save captured traffic to a .pcap file and then open the file in Wireshark for offline analysis. Alternatively, you can launch Wireshark directly and capture traffic in real time on the selected network interface. On node2, open Wireshark and select the interface you want to monitor for packet capture.

```
➤ wireshark
```

While the capture is running, use node1 to generate Ping, Telnet, and SSH traffic as performed in Task 1. Then stop the Wireshark capture. Use Wireshark display filters to locate specific packets from the captured traffic.

Questions: Apply appropriate Wireshark filters to display the following traffic. For each result, take a screenshot showing the filter used:

- Packets sent by node2 only
- Telnet traffic only
- SSH traffic only

Provide a screenshot showing that Wireshark can capture the plaintext username and password entered during a Telnet login. Explain why Telnet presents a greater security risk than SSH when used for remote access.

3. Source-IP Spoofing

Source IP spoofing modifies the source address in an IP packet so the packet appears to originate from another host. Constructing packets directly with raw sockets requires manual handling of binary packet structures, header fields, checksums, and byte ordering. In this task, use **Scapy**, a Python packet-manipulation library that allows you to construct packets by specifying protocol fields directly. This lets you focus on understanding the packet headers and spoofing behaviour rather than debugging low-level binary structures.

Your task is to create a Python program on **node1** that sends spoofed ICMP Echo Request packets to **node2**. The packets must contain a source IP address that is different from the real IP address of node1. You may use an AI assistant to help generate or troubleshoot the Scapy

	code, but you are responsible for verifying that the generated code performs the required operation correctly. Before running the spoofing program, start Wireshark on node2 and capture traffic on the interface connected to the NAT Network. After running the program, locate the generated packets and inspect their IP and ICMP headers. Take a screenshot showing the spoofed source IP address and ICMP packet information. <pre>➤ python3 spoof_icmp.py</pre> Questions: What source IP address is observed by node2? Is it the actual IP address of node1? What does this demonstrate about trusting the source IP field of an individual packet? Does node1 receive the ICMP reply to the spoofed packet? Explain where the reply would normally be sent and why. Compare the normal and spoofed ICMP packets in Wireshark. Apart from the source IP address, identify any packet fields or traffic behaviour that could potentially help a security analyst determine whether traffic is suspicious.
4.	AI-Based Traffic Analysis Scenario: You are a security analyst in the SOC. node1 (10.0.2.6) is a trusted workstation that sends operational and security information to node2 (10.0.2.7), the central monitoring server. The SOC has received a report that an attacker may have successfully gained access to the environment. At the same time, management wants confirmation that the monitored system and its security services are operating normally. However, some of the information recorded by the monitoring server does not appear completely consistent. You are given the network capture collected from the server: - lab1_incident.pcap Your task is to independently investigate the traffic and determine whether an attack occurred and, if so, reconstruct what happened. You may use AI-assisted coding to extract and process relevant information from the PCAP, and apply statistical analysis, machine learning, anomaly detection, or LLM-assisted analysis where appropriate. Your findings must be supported by evidence from the captured traffic. Question: What information or features did you extract from the PCAP for your investigation? Explain why they are relevant. Describe how you used AI (ML or LLM), or other data-analysis techniques to assist the investigation. Describe what the attacker did. What was the likely objective of the attacker? Explain how the captured traffic supports your conclusion. [Hints] - Identify useful traffic features. - Quickly scroll through packets and observe unusual changes. - Use AI to extract PCAP data into CSV. - Use ML/LLM to identify abnormal patterns. - Refer to Task 3 and consider possible spoofing.

5. Submission

- 1 PDF Report: Include screenshots, answers, analysis results.
- Code Files: Scripts used in the lab.
- Analysis Files: Include any processed data or supporting files.

Your findings and what you learned from the investigation are more important than including large amounts of raw output.

File names:

Lab1_<StudentID>_<Name>.pdf

Lab1_<StudentID>_<Name>_Code.zip